Guia Prático: Consultas SQL vs. MongoDB (PyMongo)

Referência rápida de filtros, operadores matemáticos, buscas parciais, projeções e manipulação de resultados em Python.

No MongoDB, não escrevemos comandos em texto puro como no SQL. Em vez disso, passamos **dicionários Python** para os métodos da coleção (como `colecao.find(...)`), utilizando operadores iniciados com o cifrão (`$`).

1. Tabela Comparativa Geral (CRUD e Filtros)

Considerando uma tabela SQL ou coleção MongoDB chamada `usuarios`:

Ação	Comando SQL	Comando PyMongo
Buscar tudo	`SELECT * FROM usuarios`	`colecao.find()`
Buscar apenas 1	`SELECT * FROM usuarios LIMIT 1`	`colecao.find_one()`
Filtro exato	`SELECT * FROM usuarios WHERE cidade = 'BH'`	`colecao.find({"cidade": "BH"})`
Escolher colunas	`SELECT nome, email FROM usuarios`	`colecao.find({}, {"nome": 1, "email": 1, "_id": 0})`
Maior / Menor	`WHERE idade > 18 AND idade <= 60`	`colecao.find({"idade": {"$gt": 18, "$lte": 60}})`
Diferente de	`WHERE status != 'bloqueado'`	`colecao.find({"status": {"$ne": "bloqueado"}})`
Lista de valores	`WHERE uf IN ('MG', 'SP', 'RJ')`	`colecao.find({"uf": {"$in": ["MG", "SP", "RJ"]}})`
Condição OU	`WHERE cargo = 'Admin' OR nivel > 5`	`colecao.find({"$or": [{"cargo": "Admin"}, {"nivel": {"$gt": 5}}]})`
Ordenar e Limitar	`ORDER BY nome ASC LIMIT 10`	`colecao.find().sort("nome", 1).limit(10)`
Atualizar	`UPDATE usuarios SET ativo = 1 WHERE id = 5`	`colecao.update_many({"id": 5}, {"$set": {"ativo": 1}})`
Deletar	`DELETE FROM usuarios WHERE ativo = 0`	`colecao.delete_many({"ativo": 0})`

2. Dicionário de Operadores Matemáticos e Lógicos

Como o JSON e os dicionários Python não aceitam sinais matemáticos soltos dentro das chaves (como `>` ou `!=`), o MongoDB utiliza siglas em inglês precedidas de `$`:

Operador	Significado (Inglês)	Equivalente SQL	Exemplo em PyMongo
`$eq`	Equal (Igual a)	`=`	`{"idade": {"$eq": 25}}` ou `{"idade": 25}`
`$ne`	Not Equal (Diferente de)	`!=` ou `<>`	`{"status": {"$ne": "inativo"}}`
`$gt`	Greater Than (Maior que)	`>`	`{"idade": {"$gt": 18}}`
`$gte`	Greater Than or Equal (Maior ou igual)	`>=`	`{"salario": {"$gte": 3000}}`
`$lt`	Less Than (Menor que)	`<`	`{"estoque": {"$lt": 10}}`
`$lte`	Less Than or Equal (Menor ou igual)	`<=`	`{"nota": {"$lte": 5.0}}`
`$in`	In (Está dentro da lista)	`IN (...)`	`{"uf": {"$in": ["MG", "SP"]}}`
`$nin`	Not In (Não está na lista)	`NOT IN (...)`	`{"perfil": {"$nin": ["banido", "teste"]}}`

3. Comparações Parciais de Texto (O LIKE do SQL)

Para verificar se um trecho de texto está presente dentro de um campo, o MongoDB utiliza **Expressões Regulares (`$regex`)**. A opção `"$options": "i"` torna a busca *case-insensitive* (ignora diferenças entre letras maiúsculas e minúsculas).

- **Contém o texto em qualquer posição (`LIKE '%silva%'`):**

```
# Encontra "João Silva", "silvana", "SILVA"
resultados = colecao.find({"nome": {"$regex": "silva", "$options": "i"}})
```

- **Começa com o texto (`LIKE 'ana%'`):**

```
# O símbolo ^ (circunflexo) indica o início exato do texto
resultados = colecao.find({"nome": {"$regex": "^ana", "$options": "i"}})
```

- **Termina com o texto (`LIKE '%gmail.com'`):**

```
# O símbolo $ (cifrão) indica o final exato do texto
resultados = colecao.find({"email": {"$regex": "gmail.com$", "$options": "i"}})
```

4. Contagem de Itens na Coleção (O COUNT do SQL)

No PyMongo moderno, utilizamos o método `count_documents()`, que exige sempre um dicionário de filtro como argumento (mesmo que seja um dicionário vazio `{}` para contar tudo).

```
# 1. Contar todos os documentos (SELECT COUNT(*) FROM usuarios)
total = colecao.count_documents({})
print(f"Total de registros: {total}")

# 2. Contar apenas os que atendem a um critério (SELECT COUNT(*) WHERE ativo = True)
ativos = colecao.count_documents({"ativo": True})
print(f"Usuários ativos: {ativos}")
```

5. Entendendo Projeções: O que significa 1 e 0

O método `.find(filtro, projecao)` aceita um segundo dicionário chamado **projeção**. Ele serve para escolher quais colunas o banco deve enviar de volta, economizando memória e rede.

- `1` **(Incluir / Exibir):** Diz ao MongoDB para trazer aquele campo no resultado.
- `0` **(Excluir / Ocultar):** Diz ao MongoDB para esconder aquele campo no resultado.
- **Por que usar `"_id": 0`?** Por padrão, o MongoDB *sempre* envia o campo `_id` em todas as consultas, mesmo quando você pede apenas outros campos. Colocar `0` força o banco a ocultá-lo.

```
# Exemplo: Busca todos os documentos ({}), trazendo apenas 'nome' e 'email' e escondendo o
'_id'
cursor = colecao.find({}, {"nome": 1, "email": 1, "_id": 0})

# Resultado retornado pelo banco:
# {"nome": "Carlos", "email": "carlos@email.com"}
```

Regra Importante das Projeções:

Você **não pode misturar `1` e `0`** para campos comuns na mesma consulta (por exemplo, `{"nome": 1, "idade": 0}` gera um erro). Ou você lista apenas os campos que quer ver (`1`), ou lista apenas os campos que quer esconder (`0`). A única exceção permitida pelo MongoDB é o `"_id": 0`.

6. Manipulando Resultados e Chaves no Python

Quando você executa `.find()`, o PyMongo retorna um **Cursor** (um objeto iterável). Ao percorrê-lo com um laço `for`, cada documento retornado é um **dicionário Python comum**.

A. Acessando valores e inspecionando chaves

```
for doc in colecao.find():
    # 1. Imprimir o dicionário completo
    print(doc)

    # 2. Acessar um valor de forma segura (evita erro se a chave não existir no documento)
    nome = doc.get("nome", "Sem nome")
    print(f"Nome: {nome}")

    # 3. Listar todas as chaves (colunas) presentes neste documento específico
    chaves = list(doc.keys())
    print(f"Chaves deste registro: {chaves}")

    # 4. Percorrer chave e valor simultaneamente
    for chave, valor in doc.items():
        print(f"  {chave}: {valor}")
```

B. Convertendo o resultado para uma Lista Python

Se precisar carregar todos os resultados na memória de uma vez (para verificar se a busca veio vazia ou transformar em um DataFrame do Pandas), envolva a consulta com `list()`:

```
lista_resultados = list(colecao.find({"ativo": True}))

if len(lista_resultados) == 0:
    print("Nenhum registro encontrado.")
else:
    print(f"Quantidade encontrada: {len(lista_resultados)}")
    print(f"Primeiro registro: {lista_resultados[0]}")
```

Dica de Boas Práticas:

Como o MongoDB não exige esquema fixo, um documento pode ter a chave `"telefone"` e o documento seguinte não ter. Sempre prefira usar `doc.get("telefone")` em vez de `doc["telefone"]` para evitar que seu código quebre com `KeyError`.